Topic: State Management in Flutter

First of all state management in flutter is process of handling the data (state) of applications so that the user interface reflects changes correctly. State refers to any data that can change during the execution of an app, such as user input, counters, or login status. In Flutter when state changes, the UI rebuild to display information.

① Explain briefly the following statemanagement used in flutter application development

⊛ Provider

Provider is a popular and friendly for beginners statemanagement solution in flutter. It is built on top of "InheritedWidget" and is mainly used to share and manage state across multiple widgets in an application. It helps separate business logic from the UI, making the code cleaner and easier to maintain. Provider commonly uses "ChangeNotifier", which notifies listening widget whenever the state management changes.

⊛ Riverpod

Riverpod is advanced and improved version of provider. It solves some limitation of provider, such as dependency on BuildContext. Riverpod allow global access to state without needing context, which makes it safer, more flexible and easier to test. it supports different types of providers like StateProvider, FutureProvider, and StreamProvider.

⊛ Bloc (Business Logic Component)

Bloc is statemanagement pattern that separates business logic from the presentation layer using events and states. It follow reactive programming principles and often uses streams. In Bloc, the UI sends events, the Bloc processes them, and then emits new states that update UI. This clear separation makes applications more structured and easier to maintain

⊛ Getx

Getx is a lightweight and powerful state management solution in Flutter. It provides state management, routing and dependency injection in one package. Getx uses reactive programming with observable variable, automatically update the UI when their values change. It reduces boilerplate code and is simple to use.

② Draw a table explaining the situation where each state management is applicable

Situation	Provider	River-pod	Block	Get x
Small application	Very suitable and easy to use	Suitable	Not reccommended (Complex)	Very suitable and simple
Medium Application	Suitable	Very suitable	Suitable	Suitable
Large/Enterprise Application	Difficult to manage	Suitable and scalable	Highly suitable structured	Less recommended
Team project	Good	Very Good (safe and testable)	Excellent (clear separation of logic)	Can be confusing without rules
Fast development	Basic structure	Good structure	Best choice (clean architecture	ideal (recommended) Very fast
Strict Architecture requirement	Basic structure	Good structure	Best choice (clean architecture)	Not ideal (Not recommended)

3. Explain in detail the key steps on how Provider is used. (Computer can be used to copy and paste code snippets)

Step 1. Add Dependency (pubspec.yaml)

Open pubspec.yaml and add provider after run flutter pub get to install and download dependencies.

```
# the latest version available on pub.dev. To see which dependenci
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.1
```

Step 2. Create Counter Provider Class

```
counter_provider.dart > ...
import 'package:flutter/foundation.dart';

class CounterProvider extends ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners();
  }
}
```

Step 3. Main File (main.dart)

Modify the main.dart

1. **Import required packages**

```
main.dart > MyApp > MyApp
// 1: Importing required packages
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'counter_provider.dart';
```

2. **Wrapping main() with ChangeNotifierProvider**

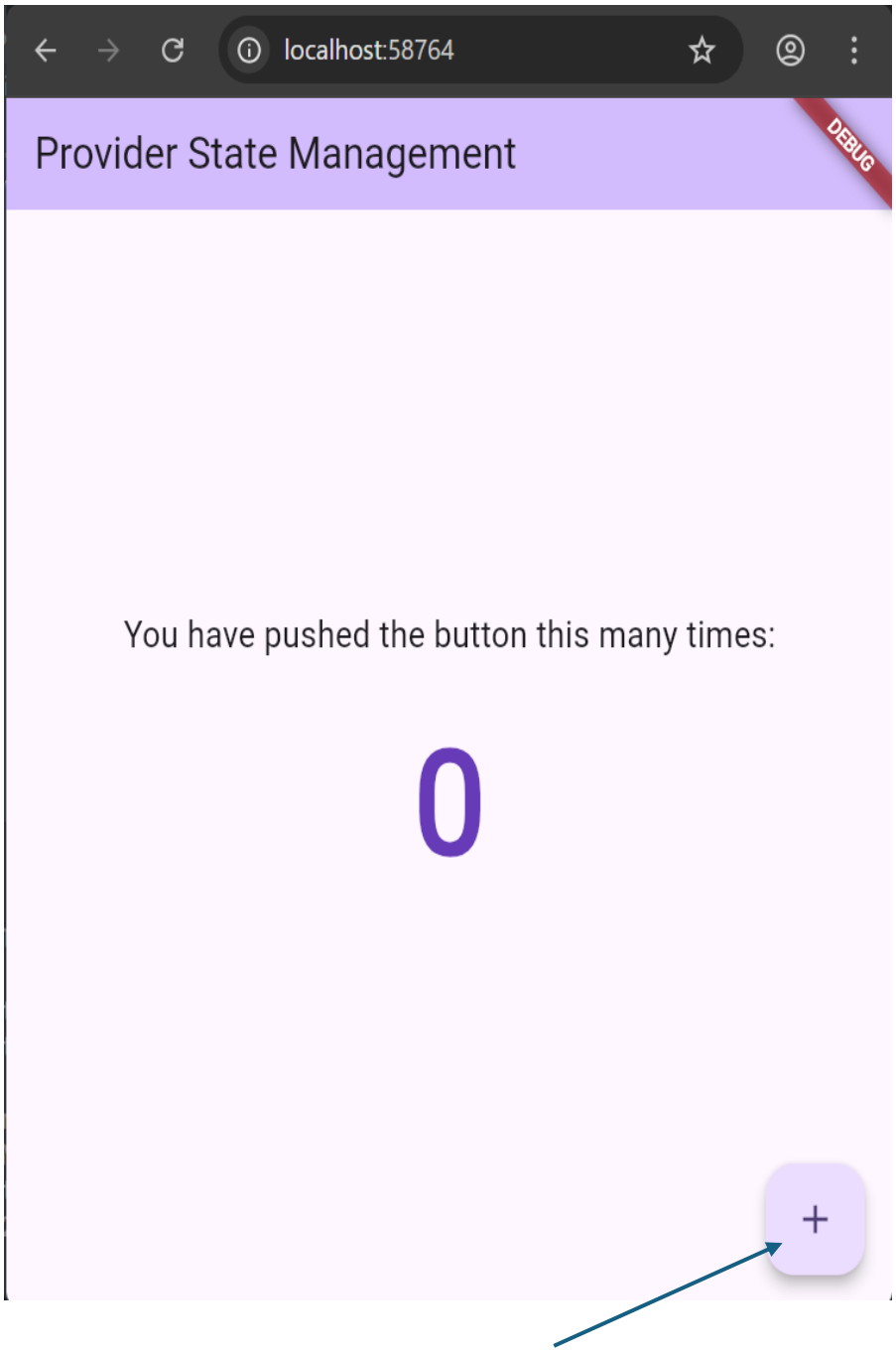
```dart
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: const MyApp(),
    ), // ChangeNotifierProvider
  );
}
```

3. **Creating UI with Consumer and Provider.**

```dart
// Crea  Declared in package:flutter_application_1/main.dart.
class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: const Text('Provider State Management'),
      ), // AppBar
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text(
              'You have pushed the button this many times:',
              style: TextStyle(fontSize: 18),
            ), // Text
```

4. **Using Consumer to display counter**

```dart
            const SizedBox(height: 20),
            //Using Consumer to display counter value
            Consumer<CounterProvider>(
              builder: (context, counterProvider, child) {
                return Text(
                  '${counterProvider.counter}',
                  style: const TextStyle(
                    fontSize: 72,
                    fontWeight: FontWeight.bold,
                    color: Colors.deepPurple,
                  ), // TextStyle
                ); // Text
              },
            ), // Consumer
          ],
        ), // Column
      ), // Center
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Provider.of<CounterProvider>(context, listen: false).increment();
        },
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ), // FloatingActionButton
    ); // Scaffold
```

Here is the output(screenshots)